D H A R

THE CANON OF DHAR

“Building toward the Holy Grail of Programming Languages”

Edition	1st Edition
Version	1.0
Date	September 5, 2026
Status	Ratified — governs Volumes I–IV
Author & Project Lead	Abdullahi Baba Garba
Scope	Index, conventions, and table of contents for the entire Canon

1. Purpose of the Canon

The Canon of Dhar is the complete set of governing documents for the Dhar programming language and its toolchain. It is organized into four Volumes, each containing a small number of named documents. This Master Index exists to do three things: state what kind of document the Canon is, list everything it contains, and fix the small number of conventions that keep four Volumes written over a long period reading as one coherent body of work rather than a pile of unrelated files.

Dhar's founding purpose, restated here because it should sit above every volume that follows: to build a systems programming language and self-hosted toolchain, from the lexer to the machine code, that gives Nigeria and Africa technological independence at the systems-software layer — and to do it by reaching for a specific, named target: the "Holy Grail" combination of C/Rust-class performance, Java-class robustness, Python-class simplicity, genuine multi-paradigm support, and full toolchain independence, all in one language.

The names in this Canon are chosen, not decorative. Dhar and Kun (its package manager) are both researched, sourced etymology, not invented flavor text — the full explanation, including what the research does and doesn't support, is in Spec0 §0.

2. What the Canon Is, and What It Is Not

Every document in the Canon is a **target specification**, not a status report. It describes what Dhar is designed to be and how it is designed to work, ratified as the standard to build against. It intentionally does not describe how close any particular day's source tree is to that standard, which files currently exist, which functions are stubs, or which bugs are open. That kind of status information is real, useful, and changes weekly — which is exactly why it belongs in separate, living audit and progress-tracking material rather than in a Canon meant to stay stable.

This separation is deliberate and load-bearing. A specification that gets rewritten every time the build is broken stops being usable as a specification. The Canon is the fixed point; the codebase moves toward it. Where the project's own prior working notes and prototypes contained real design decisions worth keeping — a keyword's meaning, a control-flow pattern, a memory-safety rule — those decisions are carried into the Canon as ratified rules. Where those notes only described the state of a particular file on a particular day, that material was left out on purpose.

3. Conventions Used Throughout the Canon

3.1 Normative language

Within any Canon document, **must** and **must not** state a hard requirement; **should** and **should not** state a strong default that may be deviated from with a documented reason; **may** states a permitted option. This mirrors the convention long used in internet and language standards, so the Canon reads as a specification to engineers already familiar with that style.

3.2 Document status

Each document carries one status: **Draft** (in active authoring, may still change shape), **Proposed** (complete and internally reviewed, open for revision before ratification), or **Ratified** (the standard in force). A Ratified document changes only through a new, numbered version with a recorded reason in its revision log — never silently.

3.3 Naming

`Dhar` (capitalized) in prose; `dhar` (lowercase) in filenames, commands, and code (`dhar build`, `.dhar` files). Cross-references cite a document and section number — e.g. “see Spec1 §8.2” — never a vague “see the docs.”

3.4 One rule, one home

Every keyword's semantics are ratified in exactly one place: Spec1. Every other document that uses a keyword links to Spec1 rather than restating or subtly re-defining it. The same principle applies across the other Volumes — a security guarantee lives in Tempering Dhar, a licensing decision lives in The Hallmark of Dhar, and everywhere else links back rather than duplicating.

4. The Four Volumes

The Canon is organized into four Roman-numbered Volumes. Volume I defines what Dhar is. Volume II defines how it gets built, stage by stage. Volume III defines how it earns trust and reaches the people meant to use it. Volume IV defines how the work of building all of the above is actually managed.

Volume	Concern	Contains
I — Vision & Specification	What Dhar is	Spec0, Spec1, Spec2…
II — Engineering Roadmap	How Dhar gets built	Stage-alpha through Stage-epsilon
III — Ecosystem, Trust & Adoption	How Dhar earns trust and reach	Six named documents
IV — Program Management	How the work is managed	The Pact of Developing Dhar

5. Volume I — Vision & Specification

Documents in Volume I are numbered `Spec0, Spec1, Spec2…` in the order a new reader should encounter them. New Spec-numbered documents may be added later — for instance a future standard-library interface spec or a binary/ABI spec — without renumbering what already exists.

Doc	Title	Covers	Status
Spec0	Overview & Vision	The problem, the thesis, architecture, use cases, a taste of the language, and how Dhar is positioned against existing languages	Ratified
Spec1	Language Specification	Full lexical, syntactic, and semantic rules: every keyword, the type system, control flow, memory model, and error handling	Ratified
Spec2+	Reserved	Future formal specifications (e.g. standard-library interface, binary/ABI) as the language matures	Not yet opened

6. Volume II — Engineering Roadmap

Documents in Volume II are numbered with the Greek sequence `Stage-alpha` through `Stage-epsilon`, one per build stage, each carrying forward the evocative name already given to that stage.

Doc	Stage Name	Mandate
Stage-alpha	Forge of Dhar	Specify the bootstrap compiler: a minimal, disposable bridge sufficient to build Stage-beta
Stage-beta	Honing Dhar	Specify the self-hosting compiler, written entirely in Dhar, that compiles itself
Stage-gamma	Optimizing Dhar	Specify the optimizing backend: SSA, register allocation, and the full optimization-pass list
Stage-delta	Gearing Dhar	Specify the standard library and tooling: build system, Kun (the package manager), formatter, LSP, and cross-ecosystem interop
Stage-epsilon	Deploying Dhar	Specify production hardening: cross-compilation, debug info, fuzzing, reproducible builds, supply-chain signing

Volume II documents specify *what each stage must achieve to be considered complete*, in the order the Thesis (Spec0 §3) requires: a stage's document should not describe Stage-gamma-level optimization concerns while specifying Stage-alpha, since the bootstrap compiler's only job is to prove the pipeline once, correctly, for a minimal subset.

7. Volume III — Ecosystem, Trust & Adoption

Six documents, each carrying a name in keeping with Dhar's forge-and-craft identity rather than a dry functional label.

Document	Covers	Primary Audience
Tooling Dhar	Editor/LSP support, documentation generator, build system, and the quality of error messages. (Kun, the package manager, is specified in Stage-delta, Volume II.)	Tooling engineers, early adopters
The Guild of Dhar	Governance: RFC process, contribution guidelines, code of conduct, decision rights	Contributors, community leads
Tempering Dhar	Security & assurance: threat model, memory-safety guarantees, supply-chain practice, certification pathways for critical-infrastructure and defence use	Security reviewers, government & defence evaluators
Casting Dhar	Adoption, education, and Pan-African reach: curricula, developer relations, hardware accessibility, government & industry partnerships	Education partners, policymakers, developer relations
Fueling Dhar	Business model, sustainability, and funding: foundation vs. company structure, licensing strategy, grants and sponsorship	Funders, co-founders, leadership
The Hallmark of Dhar	Legal, licensing, IP, and export posture: license choice, trademark, patent landscape, dual-use awareness	Legal counsel, leadership

8. Volume IV — Program Management

Volume IV consists of a single document, **The Pact of Developing Dhar**: the roadmap timeline, the risk register, and the metrics that will indicate real adoption rather than surface popularity. It is called a *pact* deliberately — it is the standing agreement, made with the project's own future work, about how progress is tracked and what counts as done.

9. Reading Paths

The full Canon is long by design — it has to be, to be complete. Nobody is expected to read it front to back. Suggested entry points by role:

If you are…	Start with
Implementing the compiler	Spec0 → Spec1 → the relevant Stage document
Evaluating Dhar for adoption or funding	Spec0 → Casting Dhar → Fueling Dhar
Assessing security or defence suitability	Spec0 → Tempering Dhar → Spec1
Joining as a contributor	Spec0 → The Guild of Dhar → the Stage document you'll work on
Tracking overall progress	The Pact of Developing Dhar

10. Amendment & Ratification

A document enters as **Draft**, becomes **Proposed** once complete and internally consistent with everything already Ratified, and becomes **Ratified** once accepted by the project lead. After ratification, a rule changes only by a new minor or major version with a one-line reason recorded in that document's revision log — the same discipline a language's own backward-compatibility promise would require of the language itself.

The Canon of Dhar — Master Index · v1.0 · September 5, 2026 — governs Volumes I through IV. Revision log: v1.0 — initial ratified structure under the Spec/Stage/named-document nomenclature.